

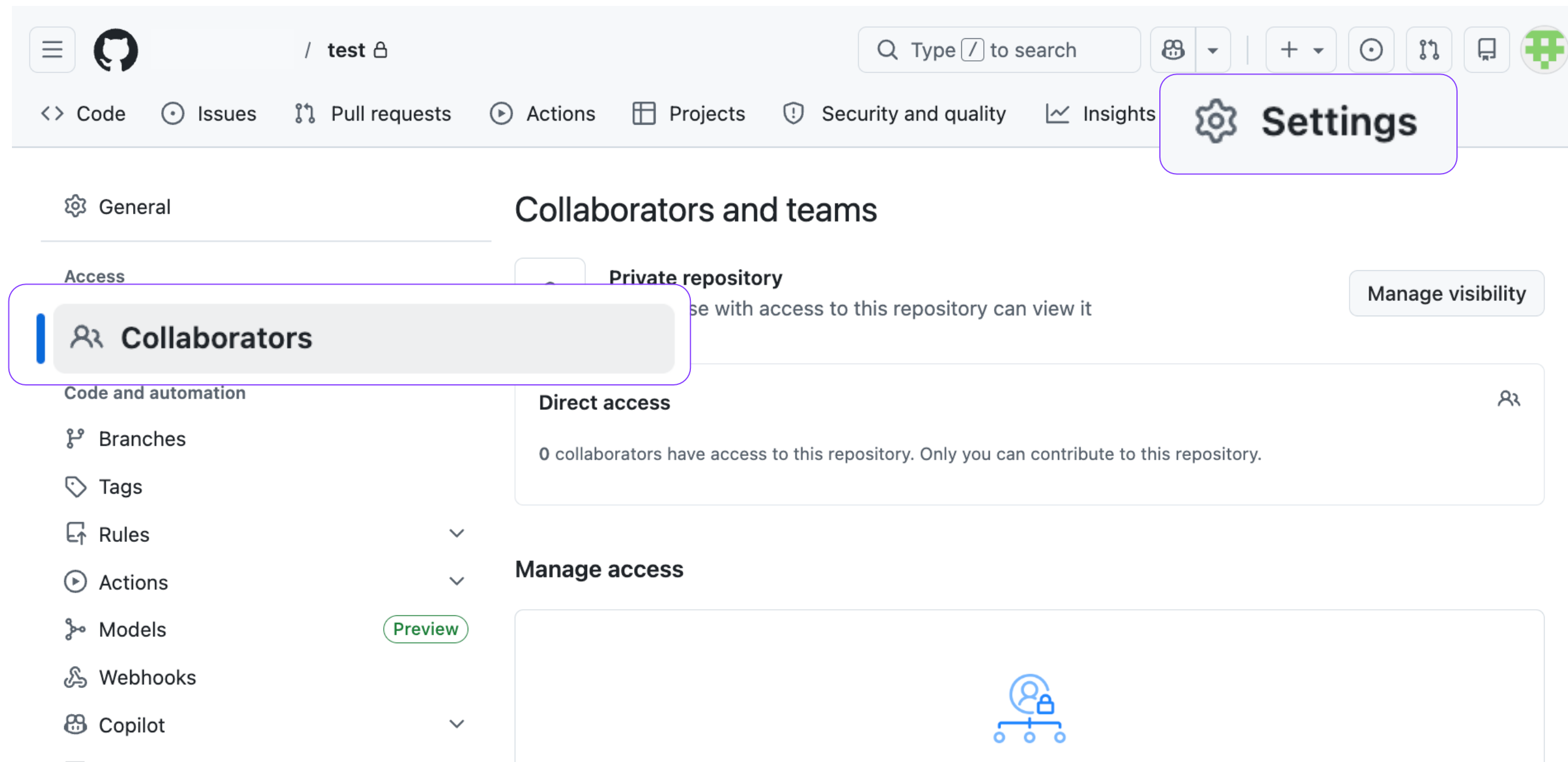
Collaborating with GitHub

GitHub 협업

협업 기초와 브랜치 전략

효율적인 협업을 위한 Collaborator

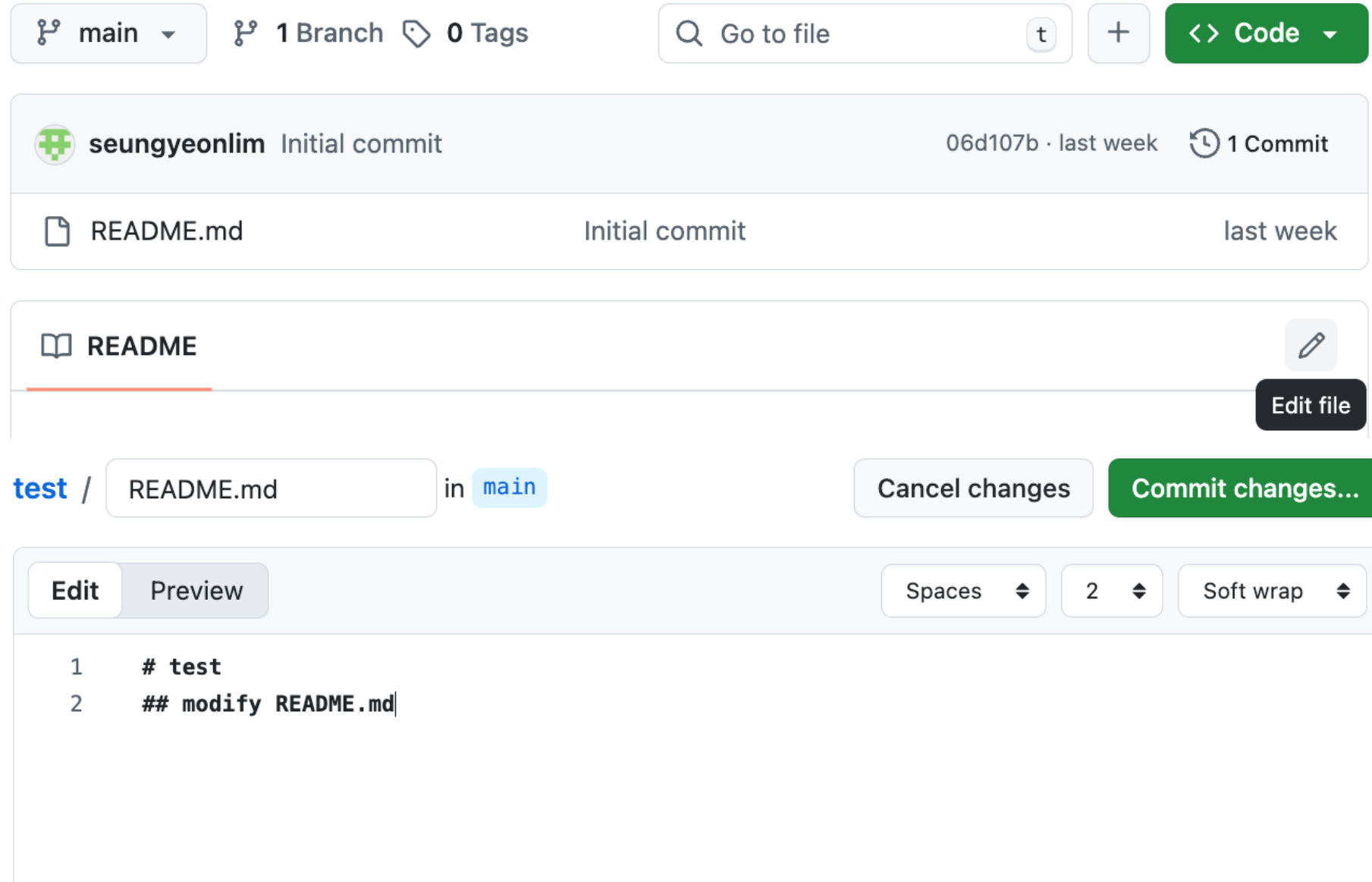
효율적인 협업을 위한 Collaborator



Collaborator(팀원) 등록방법

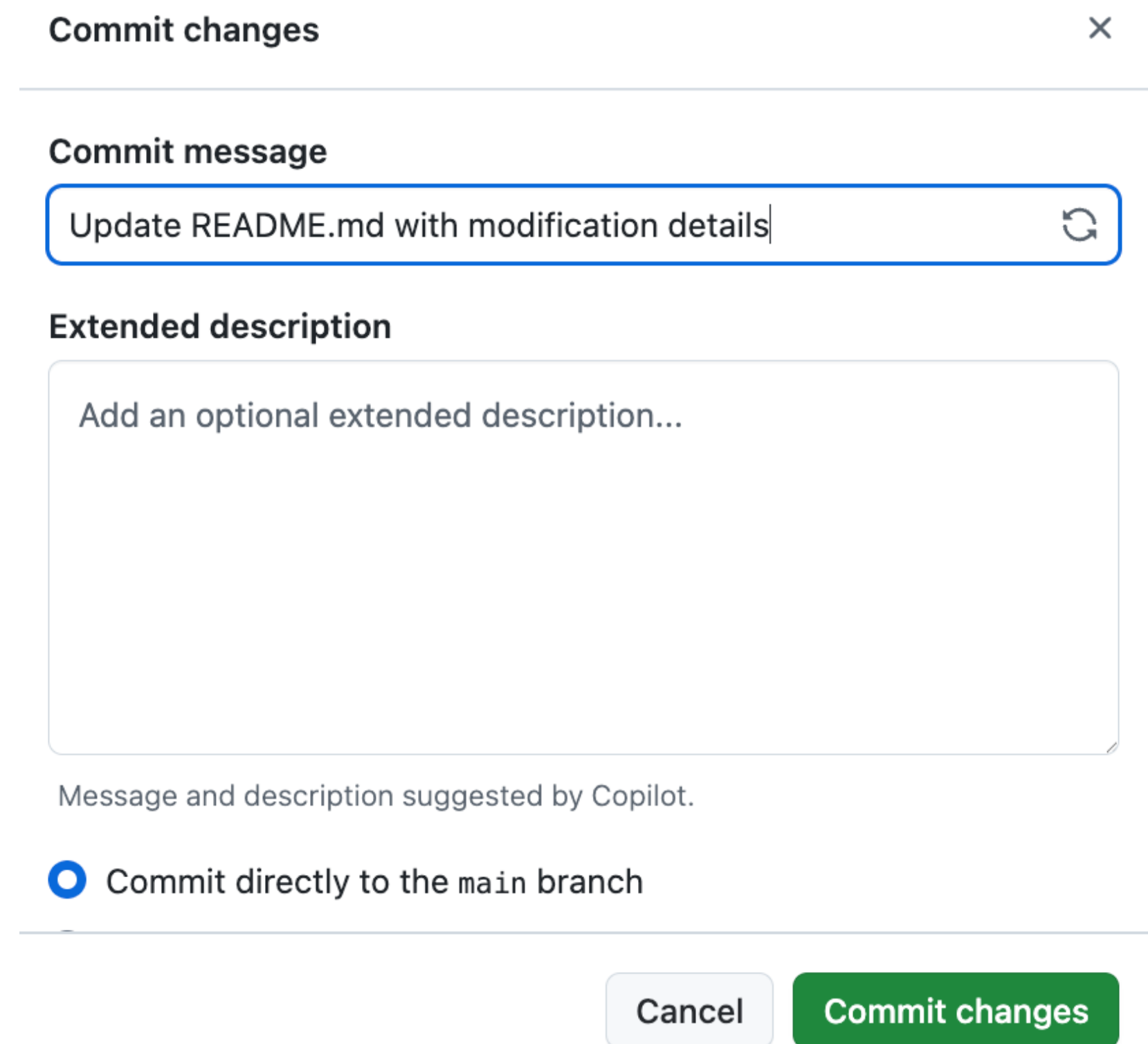
- Repository ⇒ Settings
⇒ Collaborators에서 팀원 초대
- 초대 받은 사람이 수락을 해야 함

효율적인 협업을 위한 Collaborator

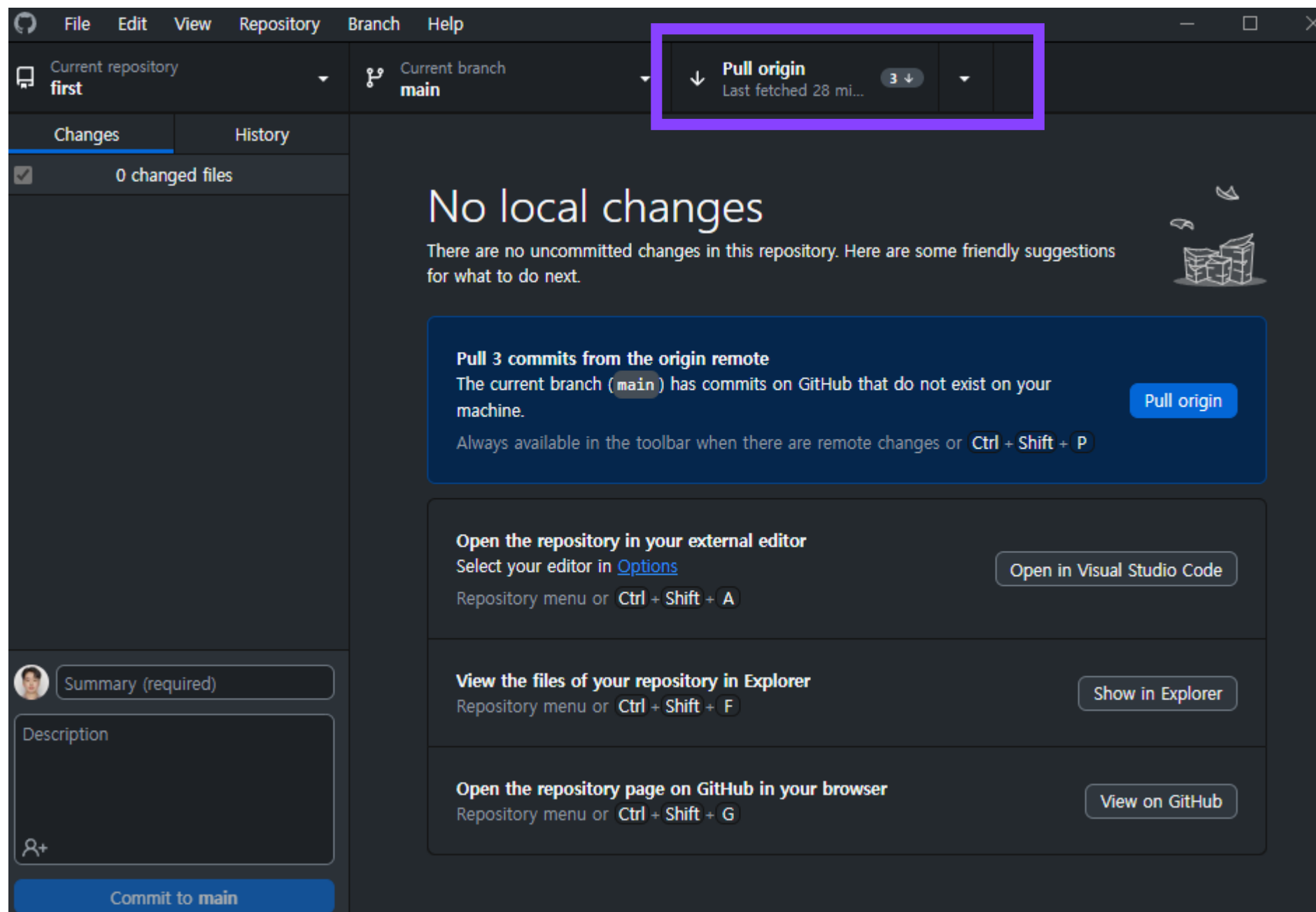


Collaborator 등록 시

- Repository를 직접 수정 가능 (연필 모양 버튼)
- 웹 상에서 바로 commit, push 가능

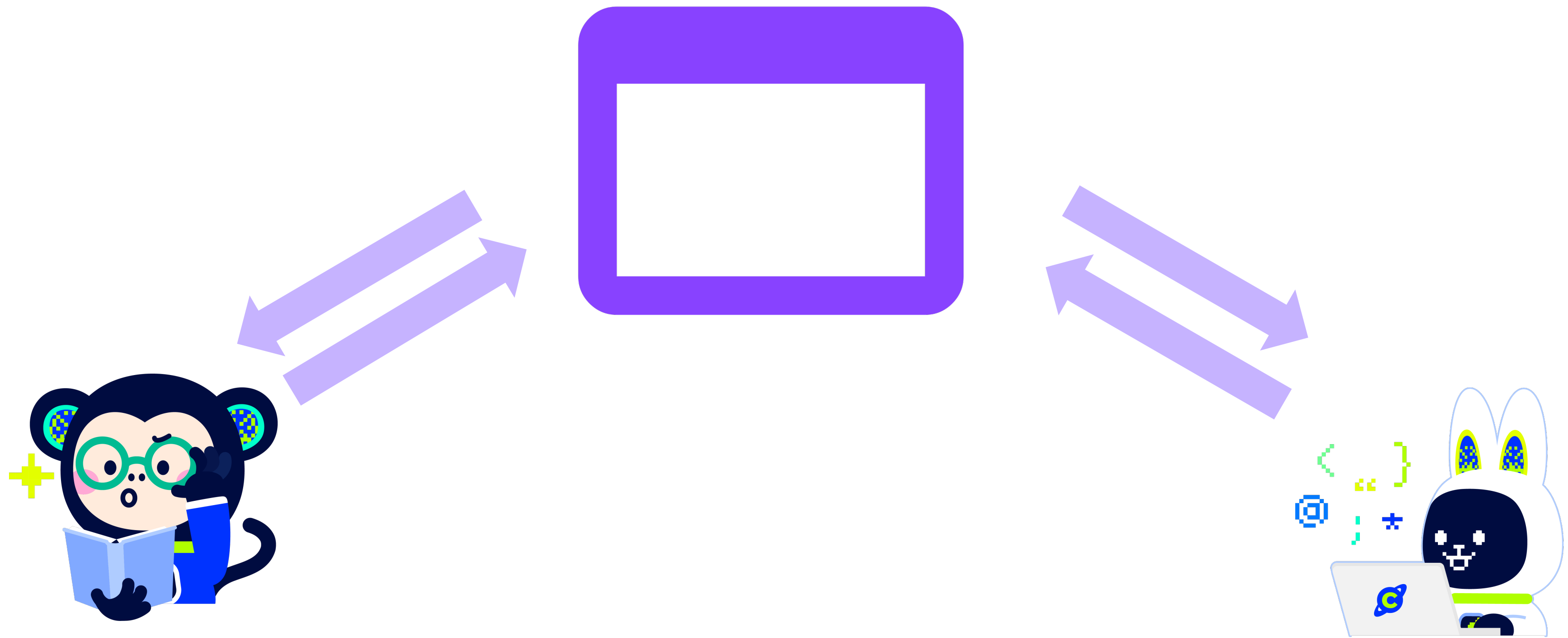


효율적인 협업을 위한 Collaborator



최신 버전 유지를 위해서,
Remote Repository의 변경 사항 로컬 적용 필수
→ Pull 필수

최신 버전이 유지되지 않을 때,
충돌이 발생하여 원활한 협업 불가능



같은 Remote Repository를 직접 수정

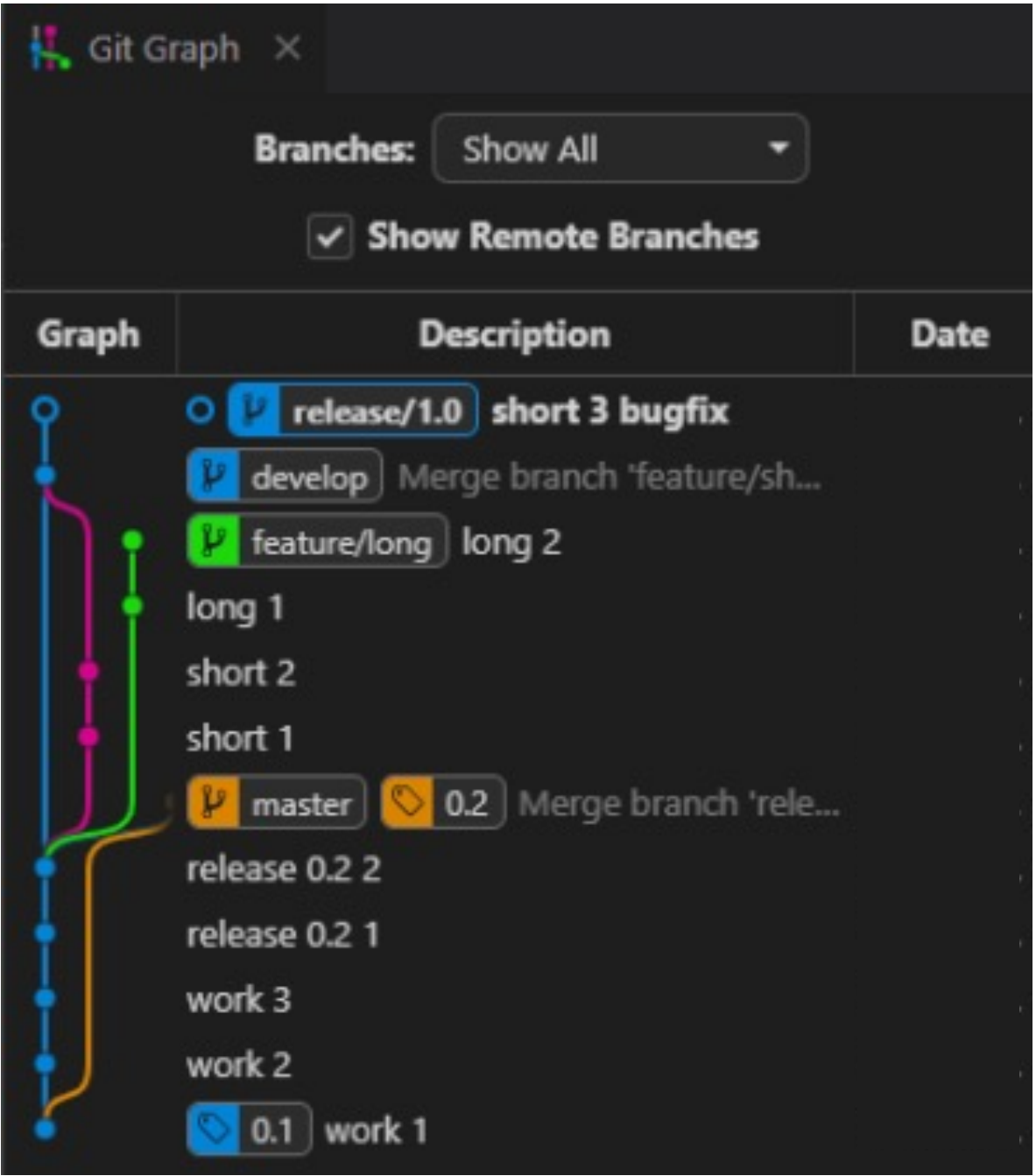
Branch 전략과 Merge

Git Flow 설계 방법

- 중대형 프로젝트에 적합한 전략으로 병렬 개발을 위한 전략으로 주로 사용이 됨
- 해당 전략은 'Vincent Driessen'이 2010년도에 제안한 방식으로 기능 개발과 유지보수를 위한 브랜치 전략을 제공함

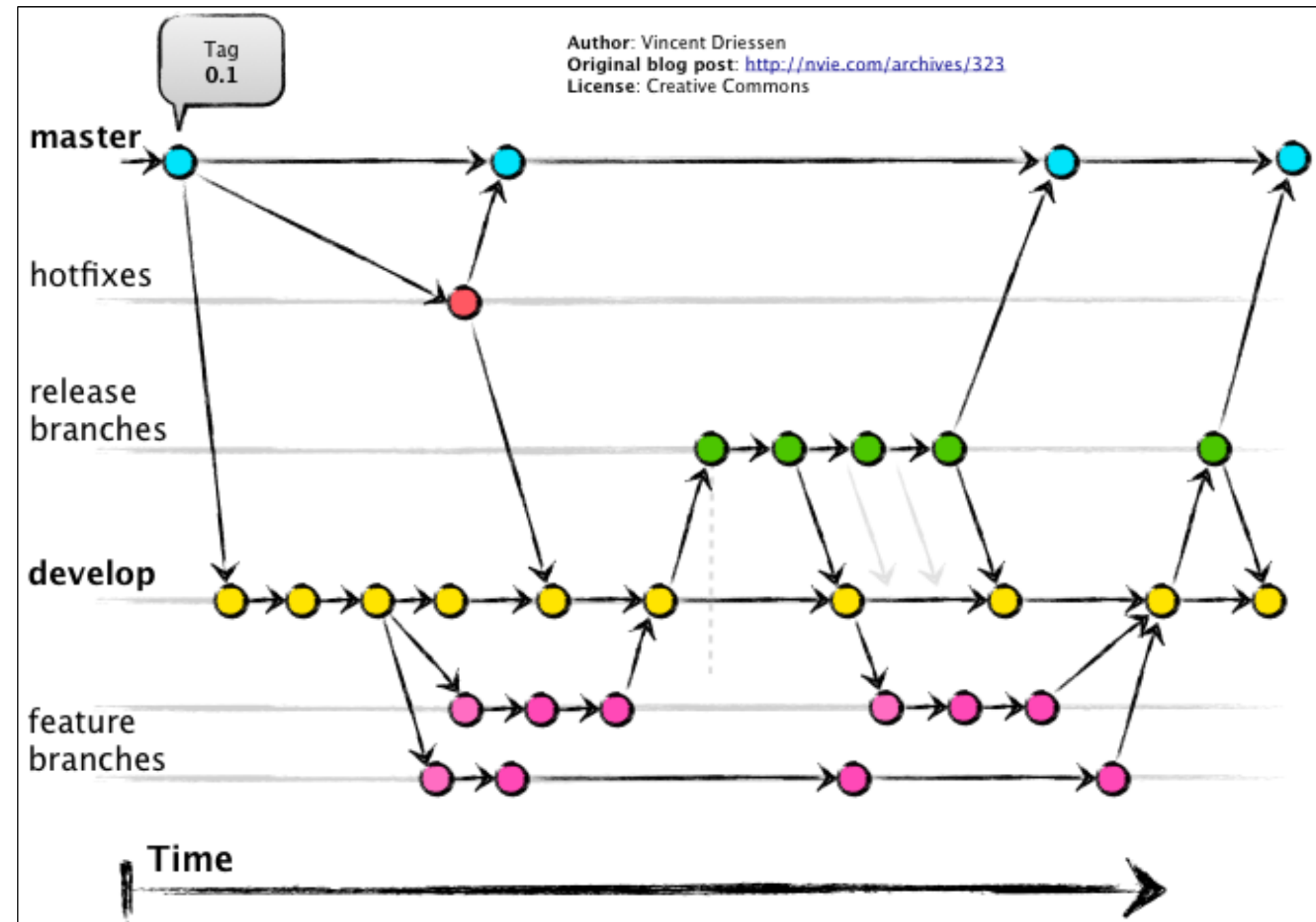
주요 브랜치

브랜치 이름	설명
main(master)	제품 출시 버전을 관리하는 브랜치
develop	다음 출시 버전을 개발하는 브랜치
feature	새로운 기능을 개발할 때 사용하는 브랜치
release	다음 출시 버전을 준비하는 브랜치
hotfix	긴급한 버그 수정이 필요할 때 사용하는 브랜치



Git Flow 브랜치의 흐름

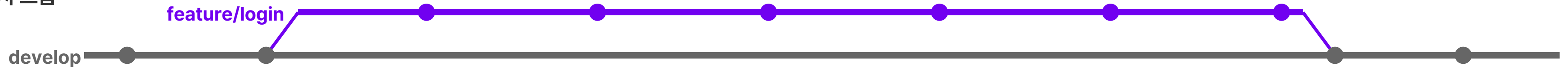
단계	브랜치	핵심 역할	흐름
1	develop	개발 최신 코드	시작
2	feature	기능 개발	develop → feature
3	feature 병합	기능 완료	feature → develop
4	release	릴리즈 준비	develop → release
5	release 병합	안정화	release → develop
6	main 배포	최종 릴리즈	develop → main
7	main	안정 코드	-
8	hotfix	긴급 수정	main → hotfix → main



실무 워크플로우 ① Feature 개발 흐름



브랜치 흐름





Branch 생성과 Pull Request

실무 워크플로우 ② Release & Hotfix 흐름

Release · 계획된 배포

1. release 분기

```
git checkout -b  
release/1.2.0  
develop
```

develop에서 컷

2. 베타 테스트

QA / 스테이징
배포 검증

기능 동결

3. 버그 픽스

```
git commit -m  
"fix: ..."
```

release에서만 수정

4. main 머지 + 태그

```
git merge  
release/1.2.0  
git tag v1.2.0
```

운영 반영

5. develop 백머지

```
git checkout develop  
git merge  
release/1.2.0
```

수정사항 동기화

Hotfix · 긴급 운영 패치

1. 운영 버그

Sentry / 고객 리포트

main에서 출발

2. hotfix 분기

```
git checkout -b  
hotfix/login-crash  
main
```

develop이 아닌 main

3. 수정 + 검증

```
git commit -m  
"fix: login crash"
```

최소 변경

4. main 머지 + 태그

```
git merge  
hotfix/login-crash  
git tag v1.2.1
```

즉시 배포

5. develop 백머지

```
git checkout develop  
git merge  
hotfix/login-crash
```

동일 버그 재발 방지



브랜치 네이밍 가이드(1/2)

- 효율적인 협업과 코드 관리의 일관성을 위해 브랜치 네이밍 규칙을 지켜야 함
- 브랜치 이름만 보더라도 목적, 작업 범위, 관련 이슈를 쉽게 파악할 수 있도록 작성함

1. 기본 규칙

- 소문자와 하이픈(-)만 사용함
- 공백과 특수문자는 사용하지 않음
- 브랜치 이름은 짧고 명확하게 작성함

2. 브랜치 유형 및 접두사(prefix)

유형	접두사 예시	설명
기능 개발	feature/	새로운 기능 개발 시 사용
버그 수정	bugfix/	일반 버그 수정
긴급 수정	hotfix/	프로덕션 환경의 긴급 버그/보안 수정
릴리즈 준비	release/	릴리즈 버전 준비 및 QA
실험 / 테스트	experiment/	실험적 기능 개발
문서 작업	docs/	문서 수정 및 추가

```
~/test-project on main
> git flow init

Which branch should be used for bringing forth production releases?
- main
Branch name for production releases: [main]
Branch name for "next release" development: [develop]

How to name your supporting branch prefixes?
Feature branches? [feature/]
Bugfix branches? [bugfix/]
Release branches? [release/]
Hotfix branches? [hotfix/]
Support branches? [support/]
Version tag prefix? []
Hooks and filters directory? [/Users/undo/test-project/.git/hooks]
```



브랜치 네이밍 가이드(2/2)

3. 네이밍 패턴

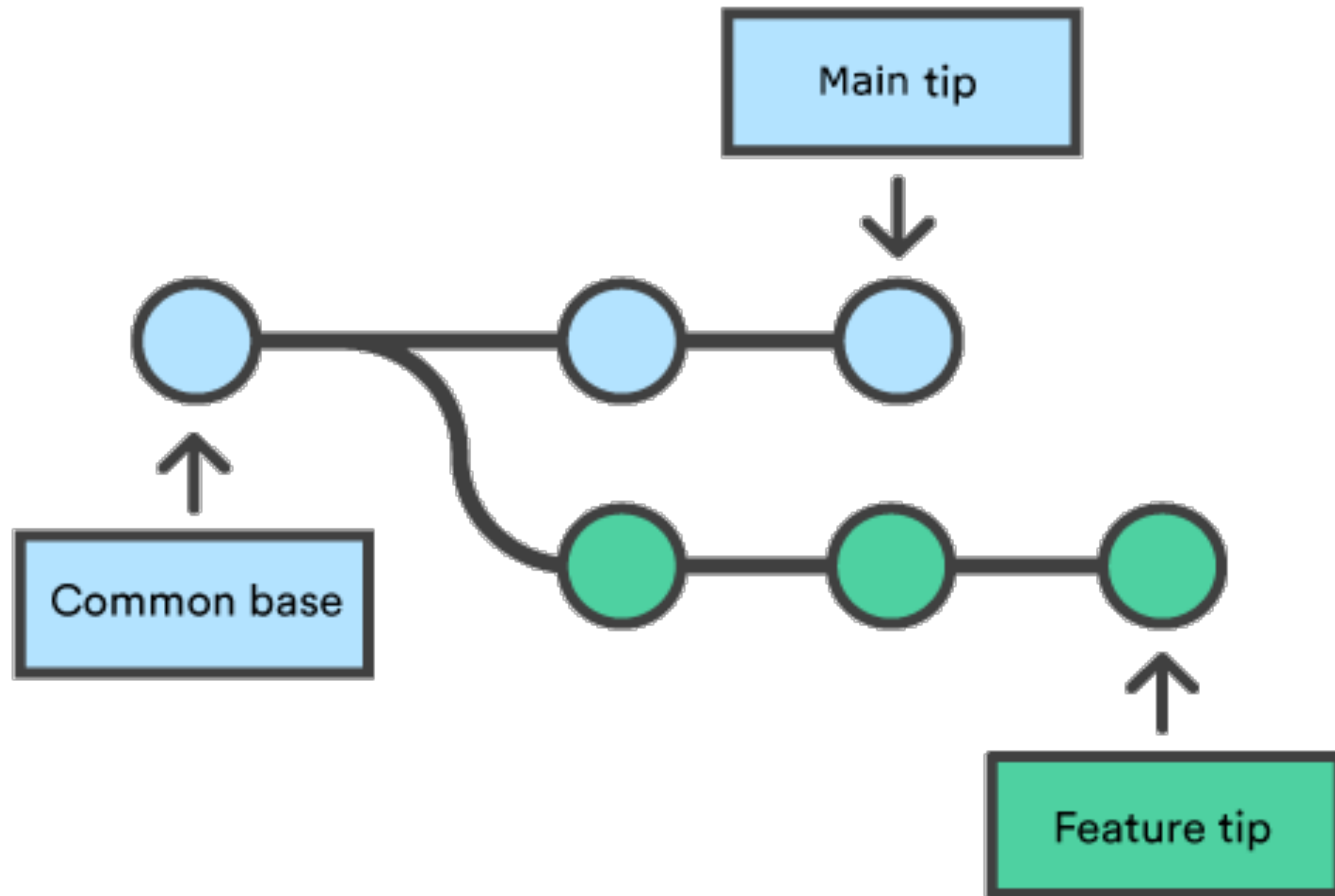
- type : 브랜치 유형 접두사(feature, bugfix, hotfix, ...)
- short-description : 작업 내용을 간결하게 설명 (하이픈으로 구분)
- Issue-number : (선택) GitHub/Jira 등의 이슈 번호

4. 예시

로그인 기능 추가	feature/login-api-#45
장바구니 수량 오류 수정	bugfix/cart-quantity-#102
서버 응답 지연 긴급 수정	hotfix/server-latency-#210
1.2.0 버전 릴리즈 준비	release/1.2.0
API 문서 업데이트	docs/api-update-#88



병합(Merge)



- 한 브랜치의 변경 사항을 다른 브랜치로 통합하는 과정을 의미함
- Git에서는 두 개의 브랜치를 하나로 합치는 작업을 수행할 때 이를 이용함
- 이러한 병합 과정을 통해서 프로젝트의 모든 구성원이 동일한 메인 코드베이스에서 작업할 수 있음

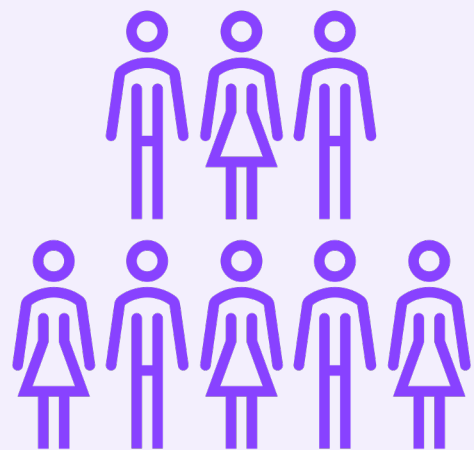


왜 브랜치 병합 전략이 필요할까?

동시 작업의 효율성 향상

독립 브랜치에서
개발/커밋 가능

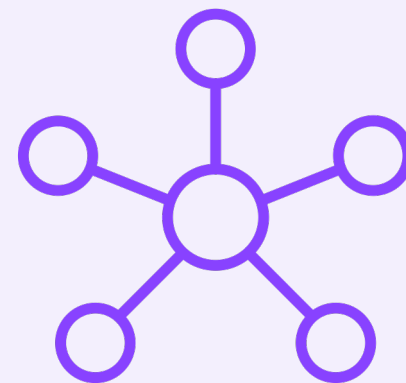
충돌은 Merge
시점에서만 해결



명확한 목적을 가진 브랜치 구조

환경별 브랜치 분리

main(배포),
develop(테스트),
feature/login(기능)



버전 관리 및 롤백의 명확성

안정 브랜치 명확 →
빠른 롤백

비상 상황에서도
신속 대응 가능





브랜치 병합 전략(Branch Merge Strategy)(2/3)

- GitHub에서 브랜치들을 메인 브랜치로 병합할 때 사용할 수 있는 규칙들을 의미함
- 각각의 전략은 서로 다른 장단점을 가지고 있어 프로젝트의 특성과 팀의 워크플로우에 따라 적절한 규칙을 선택해야 함
- 특히, 프로젝트의 소스코드 히스토리를 어떻게 관리할 것인지에 대한 팀의 합의가 중요함

병합 전략	설명	커밋 히스토리	단점
Merge Commit (기본 병합)	모든 커밋 히스토리를 보존하면서 병합하는 방식	유지됨	히스토리가 복잡해짐
Squash and Merge	여러 커밋을 하나의 커밋으로 압축하여 병합하는 방식	압축됨	작업 세부 기록은 사라짐
Rebase and Merge	커밋들을 베이스 브랜치 위로 재배치하는 선형적 방식	변형됨	충돌 발생 시 복잡, 커밋 해시 변경



브랜치 병합 전략(Branch Merge Strategy)(3/3)

- 병합 전략 설정
- 기본적으로 merge commit 방식으로 설정되어 있음
- 추가적인 병합 방법 설정이 필요하면 아래의 과정을 통해서 설정 가능함
- Github → Settings → General → Pull Requests 순서로 지정

Pull Requests

When merging pull requests, you can allow any combination of merge commits, squashing, or rebasing. At least one option must be enabled. If you have linear history requirement enabled on any protected branch, you must enable squashing or rebasing.

☒ **Allow merge commits**

Add all commits from the head branch to the base branch with a merge commit.

Default commit message

Presented when merging a pull request with merge.

Default message ▾

☒ **Allow squash merging**

Combine all commits from the head branch into a single commit in the base branch.

Default commit message

Presented when merging a pull request with squash.

Default message ▾

☒ **Allow rebase merging**

Add all commits from the head branch onto the base branch individually.